

Solving the Dog Game

Policy-Gradient and Nash-Q Approaches to a Continuous-Action, General-Sum Two-Player Game
— a Research Report

This report documents an implementation and empirical study of the "dog game" (also called the Battling Influencer Game, or AI-dog/BIG), assigned as a tractable general-sum companion problem to an ongoing grid-soccer multi-agent reinforcement learning research program. It answers the research questions raised in the project's planning meeting, using results reproduced directly from the project's own code, cross-checked against a fresh run at the time of writing.

Code and live simulator: github.com/Charith-Reddy-Pareddy/dog-game —
charith-reddy-pareddy.github.io/dog-game

All experimental numbers in Sections 5 and 6 were reproduced by re-running the project's test suite and convergence study (doggame.experiments) rather than taken on faith from prior notes; the reproduced numbers matched the previously recorded ones exactly.

1. Background and Research Program Context

The broader research program is about solving multi-agent Markov games exactly, then approximately. Prior work built an exact minimax-value-iteration solver (referred to in project notes simply as "minimax", though the more precise name is minimax value iteration) for a grid soccer game, following the classic Littman (1994) two-player zero-sum stochastic game formulation. That solver was used to study two questions that turn out to matter for everything downstream: how many game states require a genuinely mixed (randomized) equilibrium strategy rather than a pure one, and whether any equilibrium states have non-intuitive policies worth understanding by hand.

That earlier work is **planning**, not **learning**: the transition and reward functions are known exactly, so the task is to compute an equilibrium, not to estimate one from sampled experience. The project's own framing of this distinction, from the planning meeting: "we cannot even plan, learning is just an extra step ... we need to learn the game and then we need to solve the game." Two concrete findings from that exact-solver phase are directly relevant to everything in this report:

- Under a deterministic transition rule (A10-style), roughly 62 of 2,208–2,311 grid-soccer states required a mixed strategy for at least one player; under a stochastic coin-flip transition rule (Littman-style), that number rose to 94–100+. A more permissive definition of "mixed" (either player mixed, not both) raised the deterministic count further, to roughly 64–94 depending on who counted.
- The exact solver's numerical convergence threshold was tuned to $\theta = 10^{-11}$ on the value function, with the resulting Nash residual (min-max gap) on the order of 10^{-6} to 10^{-16} depending on the transition rule — i.e. genuinely at floating-point precision, not merely "small". Despite that numerical exactness, several hand-inspected "interesting" states initially showed policies that did not make intuitive sense (e.g. a blocking player moving in a direction that let the ball-carrier through uncontested); these were traced to bugs in the LLM-authored visualization and transition-rule code (two players allowed to occupy the same grid cell, in particular), not to the solver's mathematics. The methodological lesson carried into this project is explicit in the meeting notes: verify a learned or solved policy's *optimality* directly, and only trust a visualization once the underlying numbers have been checked by hand.

The dog game was introduced specifically to give the two solution approaches earmarked for soccer — DQN-style Nash-Q learning and policy-gradient self-play — a smaller, better-understood proving ground: a general-sum (not zero-sum) game with a provable uniqueness condition for its Nash equilibrium, continuous rather than discrete actions, and a closed-form solver to check any learned policy against. The soccer game offers none of those guarantees: it is zero-sum but combinatorial, its Markov structure genuinely depends on state (ball possession), and it is known from the exact-solver phase to have many states with non-unique or mixed equilibria.

2. The Dog Game: Definition and Analytical Solution

Two players, red and blue, each own a house at a fixed point in a bounded, convex domain (a unit square, by default). Each round, both players simultaneously choose a point in the domain; a shared "dog" moves to the $w / (1-w)$ weighted average of the two chosen points. Each player is rewarded by the negative squared distance from the dog to their own house. Because the game is framed in terms of continuous positions and a convex combination rather than discrete grid moves, it is structurally identical to the "battling influencer"

framing raised in the same meeting: two influencers each stake out an opinion to pull a shared audience toward them, and because audience opinion is bounded, a player who wants the audience exactly at their own position must overshoot — state an opinion further out than they actually hold — to counteract the other player's pull.

2.1 Why the equilibrium requires exaggeration

Solving each player's unconstrained best response and substituting one into the other shows there is no interior fixed point unless the two houses share a coordinate: the unconstrained recursion just pushes the action further out in one direction on every iteration. It is specifically the boundedness of the domain that turns this into a real equilibrium — both players get pushed to the domain boundary (they "exaggerate" maximally), and iterated best response converges there quickly (in two rounds, for this game, since the composition of the two best-response maps is itself a fixed point after one full alternation).

2.2 Uniqueness

The project's working conjecture, confirmed by direct testing in this codebase, is that the equilibrium is unique exactly when the two houses differ in **every** coordinate. Whenever the houses share a single coordinate (not necessarily both), that axis becomes degenerate: any pair of actions on that axis that averages back to the shared value is equally optimal, so the equilibrium set is a continuum rather than a point. The fully coincident case (identical houses) is the extreme version of this: no exaggeration is needed at all, and the equilibrium sits exactly on the shared point.

3. Methods: Two Independent Solvers

Following the meeting's split into two subgroups, this project implements both a policy-gradient solver and a DQN-style solver against the same environment and reward/transition functions, so results are directly comparable and both can be checked against the same analytical ground truth (Section 2).

3.1 Policy-gradient self-play

A single Gaussian policy module produces both players' action means via one of three network architectures, plus a learned per-player log-standard-deviation, and is trained with REINFORCE through PyTorch's automatic differentiation and optimizer — per the meeting's explicit instruction that once a network-training package is in play, no one should hand-derive gradient-descent math. Training is self-play: the same policy plays both sides, rounds are rolled out, discounted returns are computed per player, and a REINFORCE loss (log-probability of the taken action, weighted by an episode-mean-centered advantage) drives the update.

Three multi-agent network architectures were implemented behind one shared interface:

- **Separate** — two fully independent networks, one per player.
- **Shared** — a single shared trunk feeding two separate linear output heads.
- **Partial** — a shared trunk feeding two separate per-player branches (structurally equivalent to one joint network with the cross-player weights frozen to zero, but simpler to express directly as two branches off a common backbone).

3.2 Discretized DQN / Nash-Q via fictitious play

DQN needs a finite action set, so the continuous square is discretized into an $N \times N$ grid, and both players' payoff matrices over that grid are built by calling the exact same transition and reward functions the continuous environment uses — guaranteeing the discretized game and the continuous game agree on the rules by construction rather than by separate, potentially inconsistent implementations.

The general-sum stage game at each Bellman iterate is solved by fictitious play (iterated best response against the opponent's running-average strategy) rather than a linear-programming solver — the "best response dynamics" approach the meeting raised as the practical alternative to an LP solve, at the acknowledged cost of being only approximate and only guaranteed to converge for some classes of games. The outer Nash-Q Bellman backup, $Q = R + \gamma \text{NashV}(Q)$, is then iterated to a fixed point. Because the dog game's transition never depends on the incoming state, every state faces an identical one-shot game, so this backup collapses to iterating on a single pair of payoff matrices rather than looping over a state space — a simplification specific to this game's transition, not a general shortcut. This was verified directly against the closed-form consequence of that collapse (discounted value equals one-shot Nash value divided by $1-\gamma$) at two grid resolutions.

4. Verification Methodology

The meeting specified the actual check to use for whether a solved or learned policy is really a Nash equilibrium: for a mixed strategy, compute the value of every action played with positive probability — they should all be equal — and confirm no unplayed action would do strictly better. This project implements two versions of that idea: **exploitability**, a grid-search for a continuous action pair's largest one-sided improvement available to either player by deviating (used both offline and live, in the browser, on the project's demo page); and **mixed_strategy_indifference_gap**, the discrete-strategy version of the same check, with a probability floor tuned to fictitious play's known noise floor (its running average never fully forgets its random initial pick, leaving a permanent residual on the order of $1/\text{iterations}$ that is not itself an equilibrium violation).

4.1 A defect found and fixed during this work

While stress-testing this verification code against the kind of fine action grids the DQN pipeline is expected to use at higher resolution, a genuine, reproducible bug was found: the probability-floor filter used to decide which actions count as "played" can select **zero** actions once the grid has more than $1/\text{tolerance}$ actions, because a legitimately uniform mixed strategy then spreads every action's probability below the floor. That left the best-response gap computed against an empty array, silently returning positive infinity instead of a meaningful number (reproduced directly with a 1,024-action uniform strategy). The fix falls back to the action(s) actually holding the strategy's maximum probability whenever none clear the floor, keeping the check finite and usable as action-grid resolution grows — precisely the direction Research Question 6 (Section 6) pushes this codebase. A regression test covering a 1,500-action uniform strategy was added, and the full test suite (34 tests) passes after the fix. This finding is itself a small case study in the meeting's broader point: verification code needs the same scrutiny as the solvers it checks, since a broken verifier can silently validate a wrong result.

5. Experimental Results

The following results come from a full, fresh execution of the project's convergence study (doggame.experiments): 5 house/weight configurations $\times$ 3 network architectures, 1,500 self-play episodes each (15 independently trained runs), reported figures rounded for display. The reproduced numbers matched the project's previously recorded results exactly, confirming the study is deterministic given its seeds and that the published findings are not stale.

Arch.	Houses (red / blue)	w	Dist. to Nash (red/blue)	Exploitability (red/blue)
separate	(0.9, 0.2) / (0.1, 0.8)	0.5	0.0005 / 0.0004	0.00024 / 0.00017
separate	(0.8, 0.8) / (0.2, 0.3)	0.3	0.5166 / 0.0009	0.18336 / 0.02383
separate	(0.7, 0.1) / (0.6, 0.9)	0.7	0.0023 / 0.0021	0.00006 / 0.00021
separate	(0.95, 0.5) / (0.05, 0.5)	0.5	0.4925 / 0.4963	0.00013 / 0.00007
separate	(0.6, 0.6) / (0.4, 0.4)	0.5	0.0006 / 0.0004	0.00008 / 0.00005
shared	(0.9, 0.2) / (0.1, 0.8)	0.5	0.0005 / 0.0004	0.00023 / 0.00018
shared	(0.8, 0.8) / (0.2, 0.3)	0.3	0.0114 / 0.0015	0.00392 / 0.00007
shared	(0.7, 0.1) / (0.6, 0.9)	0.7	0.0021 / 0.0022	0.00006 / 0.00021
shared	(0.95, 0.5) / (0.05, 0.5)	0.5	0.4898 / 0.4951	0.00012 / 0.00008
shared	(0.6, 0.6) / (0.4, 0.4)	0.5	0.0006 / 0.0004	0.00009 / 0.00006
partial	(0.9, 0.2) / (0.1, 0.8)	0.5	0.0008 / 0.0007	0.00040 / 0.00032
partial	(0.8, 0.8) / (0.2, 0.3)	0.3	0.0020 / 0.0014	0.00081 / 0.00006
partial	(0.7, 0.1) / (0.6, 0.9)	0.7	0.0032 / 0.0046	0.00013 / 0.00041
partial	(0.95, 0.5) / (0.05, 0.5)	0.5	0.4837 / 0.4940	0.00018 / 0.00011
partial	(0.6, 0.6) / (0.4, 0.4)	0.5	0.0010 / 0.0006	0.00014 / 0.00008

Finding 1 — "can policy gradient converge?" needs a number attached to it

14 of 15 (configuration, architecture) runs reached exploitability below 0.004 within 1,500 episodes. The 15th — the separate architecture on house_red=(0.8, 0.8), house_blue=(0.2, 0.3), w=0.3 — was still at 0.183, clearly unconverged. Training that exact run 3,000 episodes further brought it down to 0.004: it was not stuck in a bad equilibrium, just slower, because one action dimension took longer to get pushed out to the domain boundary than the others. Self-play policy gradient does converge on this game, but "1,500 episodes" is not a portable number without naming the configuration and architecture it was measured on.

Finding 2 — multiple equilibria are real, not a training bug

When both houses share a y-coordinate (house_red=(0.95, 0.5), house_blue=(0.05, 0.5)), the y-axis of the stage game becomes degenerate: any pair of y-actions averaging back to 0.5 is equally optimal. The analytical solver's iterated best response lands on the "obvious" solution (both players at y=0.5 exactly);

policy-gradient training instead consistently found a boundary-cancellation solution (red near $y=1$, blue near $y=0$, $0.5 \cdot 1 + 0.5 \cdot 0 = 0.5$) in every one of the three architectures. Checking both with the exploitability metric confirms they are both genuinely unexploitable (< 0.0002) — two different, equally valid equilibria, exactly the degenerate case predicted in Section 2.2, not a training failure.

Finding 3 — architecture changed training speed, not reachability

Across the same 15 runs, all three architectures converged to equivalently low exploitability on 14 of 15 configurations; the one slow case above was slow specifically for the separate architecture, while shared and partial converged on that same configuration within the same 1,500 episodes. That is a single data point, not a general ranking, but it is evidence that parameter sharing does not obviously hurt convergence in this game and may act as an implicit coupling that transfers useful gradient signal between the two players' networks, speeding up the harder cases.

6. Research Questions and Answers

The following restates each research question raised in the planning meeting and answers it directly, distinguishing what this project's code and experiments actually establish from what remains reasoned inference or open work.

Q1. Can self-play policy gradient converge to a Nash equilibrium at all in a general-sum game?

Answered (for the dog game); open in general

Yes, empirically, on the dog game specifically: 14 of 15 (configuration, architecture) self-play runs reached exploitability below 0.004 within 1,500 episodes, and the 15th converged with roughly 3× more episodes (Finding 1). But this result should not be read as general evidence that independent policy-gradient self-play converges to Nash in general-sum games broadly — it is a known, long-standing negative result in game-theoretic learning that naive simultaneous gradient ascent by independent learners can fail to converge and instead cycle, even in simple two-player games (this is the same phenomenon that motivates optimistic/extra-gradient methods in the online-learning-in-games literature). The dog game converges reliably here because its structure is unusually favorable: each player's best response is an affine function of the other's action, and the bounded domain turns that affine recursion into a contraction toward the boundary. That favorable structure is exactly what the meeting intended the dog game to isolate before moving to soccer, where no such guarantee exists.

Q2. Does the number of self-play episodes needed to converge depend on the specific configuration?

Answered

Yes, directly demonstrated: the outlier configuration in Finding 1 needed roughly 3× the episodes of the other 14 runs. The likely mechanism is that convergence speed is governed by how far, and along how flat a gradient, the initial policy has to move to reach the boundary in each action dimension — a geometric property of the specific house positions and w , not a property of the algorithm or architecture alone. This means any single episode-count recommendation for this game class should be reported with the configuration it was measured on, not as a universal constant.

Q3. Does pre-training help, and if so, on what does the benefit depend?

Open

Not tested in this project; no experiment here initializes a policy from a related configuration before self-play. Reasoned expectation: the dog game's equilibrium action is a simple, smooth (piecewise affine, clipped) function of the house coordinates and w , so a network pre-trained by direct regression against `doggame.nash.solve_stage_nash` across many sampled configurations (an amortized-solving approach) should need few or zero further self-play episodes to reach a low-exploitability policy on a new configuration, and should specifically help the slow outlier case in Finding 1 by giving it a warm start already near the boundary. This is a concrete, low-cost next experiment this codebase is already positioned to run.

Q4. When multiple Nash equilibria exist, which one does self-play find, and is selection consistent?

Partially answered

In the one degenerate configuration tested (shared y-coordinate), policy gradient consistently selected the same boundary-cancellation equilibrium across all three network architectures, rather than the solver's interior "obvious" solution (Finding 2). This is consistent with, though not conclusive proof of, a specific mechanism: the policy's log-standard-deviation is initialized at -0.5 ($\text{std} \approx 0.61$, large relative to the unit domain), so early sampling routinely lands near the domain boundary and gets clipped there; because the degenerate axis has zero reward gradient everywhere on the indifference line, nothing in training ever pulls the policy back off the boundary once it lands there, so the initial exploration noise combined with clipping — not a genuine equilibrium-selection principle — likely decides the outcome. Whether the same equilibrium is also selected consistently across different random seeds (as opposed to just across architectures) was not tested and would be needed to confirm this explanation.

Q5. How should the stage game inside the Nash-Q Bellman backup be solved, and how much approximation error is tolerable?

Partially answered

This project solves it with fictitious play rather than an LP solver, and directly verifies that the resulting fixed point matches the closed-form value the dog game's state-independent transition implies, at two grid resolutions (5 and 9). That confirms fictitious play's approximation error is small enough not to matter for *this* game. It does not answer the general question the meeting posed — how much stage-game solution error the outer Bellman iteration can tolerate before it stops converging to the right fixed point. That is a genuinely under-studied question: Nash-Q's own convergence guarantees (Hu and Wellman, 2003) already require restrictive conditions on every stage game visited during training (a global optimum or a saddle point at every iterate), and substituting an approximate solver for the exact one visited at each iterate could break those conditions in games less structurally forgiving than the dog game. This remains open.

Q6. How does discretized-action resolution trade off against solution quality, and is a function approximator needed at scale?

Partially answered

The payoff-matrix construction is $O(N^4)$ in grid resolution N (an N^2 -point grid produces an $N^2 \times N^2$ payoff matrix), so resolution scales very poorly as a brute-force approach. The verification bug described in Section 4.1 is a direct, concrete instance of this question: it only manifests once resolution passes roughly 32 (1,024 actions), i.e. exactly the regime this question asks about, and was found and fixed as part of this project rather than being a hypothetical concern. Beyond that, a full sweep across resolutions was not run; the meeting's own instinct — that a neural-network Q-function approximating over actions directly will be necessary at practical resolution — is not contradicted by anything found here, but is not yet tested in this codebase either.

Q7. What is the right multi-agent policy network architecture: separate, shared, or partially shared?

Partially answered

On the dog game, architecture changed training *speed* but not *reachability*: all three architectures converged to equivalently low exploitability on 14 of 15 configurations, and on the one hard configuration, shared and partial converged within the budgeted episodes while separate needed roughly 3× more (Finding 3). A plausible mechanism is that a shared trunk lets gradient signal from either player's loss update representations useful to both, effectively giving each player more informative updates per episode than a fully independent network gets. This is a single game's worth of evidence, though, and the meeting's

broader question — whether this generalizes, and how these architectures perform specifically *against each other* (cross-play, not just self-play) rather than only against themselves — is untested here.

Q8. When is the dog game's Nash equilibrium unique?

Answered

Confirmed directly: uniqueness holds when the two houses differ in every coordinate, and fails independently on each coordinate the houses happen to share — each shared coordinate opens one degenerate direction in the action space along which a continuum of equally optimal action pairs exists (Finding 2, Section 2.2). The fully coincident case (identical houses on every coordinate) is the extreme instance: the equilibrium set collapses to the single point where no exaggeration is needed at all.

Q9. How should a solved or learned policy be verified as an actual Nash equilibrium?

Answered

Implemented exactly as the meeting specified: for continuous actions, grid-search for the largest one-sided improvement either player could get by deviating (exploitability, near-zero at equilibrium, up to grid resolution); for a discrete mixed strategy, check that every action played with positive probability has equal value and no unplayed action does better (`mixed_strategy_indifference_gap`). The defect described in Section 4.1, found by deliberately stress-testing this exact code against a realistic edge case (a fine action grid), is itself part of the answer: a verification routine is only as trustworthy as its own test coverage, and this project's original implementation would have silently returned an unusable (infinite) result rather than a wrong but plausible-looking one — a lucky failure mode, since a quieter bug (e.g. one producing 0/0 as an uncaught NaN compared as always-false) could have silently validated an unconverged policy instead.

Q10. How much of a solver's non-"clean" mixed probabilities (not exactly 50/50, 1/3-2/3, etc.) is real mixing versus numerical noise?

Open

Accounted for but not resolved: the indifference-gap check's probability floor (tol) is explicitly tuned to fictitious play's known noise floor, which decays as roughly $1/\text{iterations}$ from its random initial pick, so genuine mixing (typically far above 1% probability) is not confused with that residual. No rounding or cleanup pass exists to recover exact fractions the way the meeting's soccer-game discussion called for (e.g. a mixed strategy that should be exactly 1/3–2/3 but is reported as 0.339/0.661). A structural fix — explicitly excluding the random initial pick's weight from the running average, rather than merely tolerating its residual — would remove this source of noise rather than filtering around it, and remains future work.

Q11. Which of these findings are expected to transfer to the soccer game, and which are dog-game-specific artifacts?

Open

Largely open, and the codebase is explicit internally about the one simplification least likely to transfer: the Nash-Q Bellman backup here collapses to iterating a single pair of payoff matrices only because the dog game's transition never depends on the incoming state. Soccer's transition genuinely depends on state (ball possession, player positions), so the DQN/Nash-Q code in this project would need to be extended to loop over states and iterate value iteration properly — structurally returning to the exact grid-soccer solver's own approach (minimax value iteration) rather than reusing this project's shortcut. The architecture finding (parameter sharing not hurting, and plausibly helping on harder cases) may or may not transfer: soccer's state and action spaces are far larger and more combinatorial, where a shared trunk's limited capacity could

become a bottleneck rather than a helpful coupling. The degenerate-equilibrium phenomenon (Finding 2) is specific to the dog game's affine reward structure (shared house coordinates create a literal continuum of ties); soccer's own mixed-strategy states, per the exact-solver phase, arise from a different mechanism — symmetric threats on a discrete grid — and are not expected to reduce to the same kind of coordinate-degeneracy argument.

7. Summary of Open Questions and Recommended Next Steps

- **Pre-training (Q3):** regress a policy network directly against `doggame.nash.solve_stage_nash` across sampled house/weight configurations, then measure self-play episodes-to-converge with and without that warm start, specifically on the Finding-1 outlier configuration.
- **Equilibrium-selection consistency (Q4):** repeat the degenerate (shared-coordinate) configuration across many random seeds, not just across the three architectures, to determine whether the boundary-cancellation solution is a genuine attractor of this training setup or an artifact of one seed's initialization.
- **Stage-game solution quality vs. outer convergence (Q5):** deliberately degrade the fictitious-play sub-solve (fewer iterations, coarser tolerance) and measure how the outer Bellman fixed point's distance to the closed-form answer grows, to get an empirical error-tolerance curve where no theoretical one exists.
- **Resolution sweep (Q6):** now that Section 4.1's fix removes the correctness blocker, systematically sweep discretization resolution and measure both solution quality and wall-clock/memory cost, to locate the point where a neural-network Q-function becomes necessary rather than optional.
- **Cross-play architecture comparison (Q7):** train each architecture in self-play, then evaluate them against each other (not just against themselves) to test whether shared-trunk training produces policies that are still robust against an independently trained opponent.
- **Clean mixed-strategy recovery (Q10):** modify fictitious play to exclude the random initial pick's weight from the running average (rather than tolerating its residual via a tuned floor), and test whether that recovers exact textbook fractions on known games (e.g. matching pennies, already used as a unit test fixture in this codebase).
- **Extension to state-dependent transitions (Q11):** generalize `doggame.dqn`'s Bellman backup to loop over states properly, as a direct rehearsal for reusing this codebase's Nash-Q machinery on the soccer game, where the state-independence shortcut does not apply.

8. Relationship to the Soccer Game Research Program

The dog game was never the end goal; it is the controlled setting in which this project's two solvers, three network architectures, and verification methodology could be built and validated against a known-unique closed-form answer before being pointed at the soccer game, where no such answer exists and where the exact-solver phase already found a non-trivial fraction of states (roughly 3–5% under a deterministic transition rule, more under a stochastic one) requiring genuinely mixed equilibrium strategies. The concrete handoff points this report identifies for that next phase are: (1) the verification functions in `doggame.verify` are already game-agnostic and can be pointed directly at soccer payoff data; (2) the three network architectures and the self-play training loop carry over with only the environment swapped, since `doggame.env.DogGameEnv` was deliberately built with the transition and reward functions injected rather than hardcoded, exactly so a different game could reuse the same loop; and (3) the Nash-Q Bellman backup will need the state-loop generalization described under Q11 above before it can be applied to soccer at all.

Sources: the project's planning-meeting transcript; handwritten planning notes (soccer/dog-game session 3); and this repository's own code, tests, and a fresh re-run of `doggame.experiments` at the time of writing.